

NagarVaani — नगरवाणी

Your Voice, Your City

Project Document · Build with AI: Code for Communities (H2S / GDG) · Track 1: AI for Digital Public Infrastructure & Governance

Contents: 1. What the app does · 2. Reality matrix (real / demo / stub) · 3. Challenge alignment & expansion · 4. Security notes

NagarVaani — What the App Does

NagarVaani (नगरवाणी) — "Your Voice, Your City." A civic-intelligence layer between citizens and government. Built for *Build with AI: Code for Communities* (Hack2Skill / GDG), Track 1 — AI for Digital Public Infrastructure & Governance (BRICS theme: Innovation).

The problem

Citizen development requests live in fragmented channels (helplines, WhatsApp groups, paper letters, portals). Officials can't see the whole picture, ranking is by whoever shouts loudest, and small villages are drowned out by large cities.

The idea in one line

Low votes ≠ low importance. A monsoon drain with 3 votes outranks a dry-season pothole with 104.

Who uses it

Person	What they do
Citizen (signed in or anonymous)	Reports a problem by web form, voice note (mic) or Telegram bot, in Hindi / Marathi / Tamil / English (or Hinglish); adds photos; tracks status; votes; comments; shares; reports abusive posts
Official (ward / city / state / central; admin-approved)	Sees a queue sorted by real priority, AI briefs, hotspot map, marks in-progress / resolved, flags infrastructure gaps, reviews citizen reports, sees ranked "priority projects"
Admin	Approves or rejects official accounts; can load external datasets (population, infrastructure index, planned investment)

How a complaint travels

1. **Intake** — web form, mic recording (speech-to-text via Whisper), or Telegram text/voice. Any language.
2. **Filter (AI)** — rejects political / communal framing and suggests a neutral rephrasing.
3. **Classify (AI)** — category, language, English translation (Romanised Hindi is translated too), location hint, safety-risk yes/no.
4. **Score (pure math, no LLM)** — the L1–L5 hierarchy → 0–100 priority. See [HOW_SCORING_WORKS](#) below and the in-app page "**How scoring works (L1–L5)**".
5. **Brief (AI)** — a 2–3 sentence official brief + one recommended action.
6. **Public thread** — like a Q&A thread: comments, "same issue in my area" links (cross-district pattern → L5), place names in comments are auto-detected and linked by NLP.
7. **Action** — officials update status; the citizen is notified (email; Telegram if linked); the citizen can **dispute** a "resolved" status that isn't really fixed.

Pages

Landing · Home (ward feed) · Trending (nationwide) · Near me (GPS radius + map + email alerts) · My complaints · My votes · Complaint detail (score breakdown, photos, comments, translate, share, flag) ·

Submit · Track by ID · Report corruption (login-gated, never in public feeds) · Govt schemes directory · **How scoring works (L1–L5)** · About · Our impact · Get involved · For officials · Official portal (Queue, All, Hotspot Map, **Priority Projects**, Investment Flags + Citizen reports, Resolved, Verifications).

What the L1–L5 levels mean (plain language)

Level	Meaning	Effect
L1 Safety	Could someone be hurt within days?	+40 and a score floor of 90
L2 Season	Same problem, worse season (monsoon drains, summer water/power)	multiplier $\times 1.0$ – $\times 2.0$
L3 Problem type	Water 28 · drainage 26 · electricity 22 · tree 20 · corruption 18 · garbage 16 · road 14 · other 10	+ points
L4 People affected	Log-scaled population (max +15)	+ points (default population 10,000 until a dataset is loaded)
L5 Pattern	Same issue reported from other areas	+5 per area, max +20
Votes	Weakest signal	$\times (1 + 0.1 \cdot \log_{10}(\text{votes}+1))$

score = $\min(100, (L1+L3+L4+L5) \times L2)$, floored at 90 if L1; then $\times$ vote nudge.

What the buttons do (new in this revision)

- 🔗 **Share** — real: opens a dialog with the device share sheet (where supported), Copy link, WhatsApp, Telegram, X, Facebook, Email. Link format `.../#complaint/<id>` opens that complaint directly.
- 🚩 **Flag** — real: signed-in citizens report a post (spam / misleading / duplicate / abusive / wrong location + note). It is stored in the database and appears in the **official portal** → **Investment Flags** → **"Citizen reports"** table for that official's jurisdiction. Nothing is auto-hidden; a human reviews.
- 🚩 **Flag for investment** (officials, in the queue) — a different flag: posts an `[OFFICIAL FLAG]` note that lists the complaint under **Investment Flags** as an infrastructure gap for a senior reviewer. Only verified officials can create these.

Tech

FastAPI + SQLAlchemy (async) · SQLite locally / PostgreSQL on Render · vanilla-JS ES modules front end (Leaflet + OpenStreetMap) · Gemini by default (Claude switchable in one line) · Whisper STT (Hugging Face default, Groq/OpenAI optional) · Cloudinary images · Telegram bot · Gmail SMTP · Google Sign-In (optional).

Reality Matrix — what is real, what is demo, what is a stub

Legend:  **Real** (live data / working end to end) ·  **Real but simplified** ·  **Demo / example content** ·  **Provision built — needs data or a key** ·  **Not built**

1. Data shown to users

Area	Status	Notes
Complaint feeds (Home, Trending, Near me, My complaints, My votes)		Live DB queries. Counts, scores, votes are real.
Right-panel / dashboard counters, landing stats, nationwide stats		Computed from the complaints table (<code>/stats/*</code>).
Hotspot map points		Real geotagged complaints.  Fallback: if the DB has no geotagged complaints (or the API is unreachable) <code>js/govt.js</code> shows 4–7 hard-coded example pins (Barmer, Pune, Amravati...) so the map isn't blank — clearly a demo fallback.
Static feed cards before first load		Placeholder cards replaced by real data once the API answers; voting on them shows "Demo card".
Right-panel "My complaints" sample rows (Lane 4, streetlight...)		Static HTML sample shown before/without login.
Seed accounts & 5 seeded complaints (<code>seed_data.py</code>)		<code>ramesh@test.com</code> , <code>official@test.com</code> , <code>voter*@seed.nagarvaani</code> (~110 fake voters). Do not run the seed in a real deployment , or change/remove those accounts.
Get-involved page: NGO/CSR partners, events, before/after gallery		Marked with DEMO ribbons + banners. No real partners exist.
Our-impact "trees planted", "recycled" blocks		"Coming soon" placeholders (no invented numbers). Resolved/critical counts are  real.
Govt schemes / ministries directory		Curated India-only static list with real ministry links; not a live scheme API.
Complaint text, votes, comments, images, status log		Whatever users submit.

2. AI behaviour

Feature	Status	Notes
Political/communal filter		LLM call; fails open (passes the complaint) if the LLM is down.
Classification, language detection, translation to English		LLM; fails open to category <code>other</code> . Romanised Hindi ("Hinglish") is now detected and translated (fixed this revision).
On-demand translate (complaint / comment / card)		Now returns a clear error (503) instead of silently echoing the same text when the LLM fails. Rate-limited.
Priority score (L1–L5 + votes)		Deterministic Python, no LLM. L1's <i>input</i> (is it a safety risk?) is an LLM judgement.
L2 season		Uses the server's calendar month at scoring time (India season table); stored, not re-evaluated as seasons change.

Feature	Status	Notes
L4 population		Default 10,000 for everyone unless an admin loads a <code>population</code> dataset for the city (<code>/api/insights/indicators</code> , <code>load_indicators.py</code>). No census data is bundled. Telegram-sourced complaints still use the default.
L5 cross-area pattern		From the "Same issue in my area" button and NLP place detection in comments.
Official brief + recommended action		LLM text; fallback template if LLM fails. No ₹ cost figures (removed on purpose; none invented).
Chatbot (home)		Grounded only in a fixed fact sheet + live counts; rate-limited.
Speech-to-text		Works with a Hugging Face / Groq / OpenAI key; without one, voice returns an error. HF free tier has cold starts.

3. Buttons — which clicks are real

Click	Status
Vote (▲), Same issue in my area, Comment, Dispute, Translate, Track by ID	 real API calls
Share	 real (Web Share API + copy + WhatsApp/Telegram/X/Facebook/Email intents; deep link opens the complaint). <i>Was a fake "Link copied" toast before.</i>
Flag (citizen)	 real — POST <code>/complaints/{id}/flag</code> , stored, shown to officials. <i>Was a fake toast before.</i>
Flag for investment (official)	 real — creates an <code>[OFFICIAL FLAG]</code> comment; lists under Investment Flags.
Mark in progress / Resolve (official)	 real, now limited to the official's own jurisdiction; notifies the citizen by email/Telegram.
Submit complaint (text, mic, photos)	 real (photos need Cloudinary keys; mic needs an STT key).
Neighbourhood email alerts	 real (needs SMTP; otherwise printed to server log).
Sign-in with Google	 shown only when <code>GOOGLE_CLIENT_ID</code> is set.
Volunteer / org-partner forms	 <code>mailto:</code> style CTAs — open the visitor's mail app; nothing is stored server-side.
Get-involved "shout-out" / gallery / event cards	 example layout only.
Gallery thumbnails with emoji in the static detail template	 replaced by real images when a complaint has any.
Help & FAQ modal, Chatbot	

4. Channels

Channel	Status
Web form	
Mic voice note	 (STT key)
Telegram bot (text + voice, account linking, status replies)	 (bot token + webhook secret). Conversation state is in memory — lost on restart.
WhatsApp	 Not built. The DB column <code>source_channel</code> allows <code>whatsapp</code> , but no integration exists (needs WhatsApp Cloud API + business approval). Telegram is the built messaging channel.
SMS / IVR	

5. Challenge coverage & where to expand

Challenge element	Status	Where it plugs in
Aggregate citizen requests via voice, text, messaging apps	✅ web / mic / Telegram · ❌ WhatsApp	<code>routers/telegram.py</code> is the template for a WhatsApp Cloud API webhook
Multilingual	🟡 UI: EN/HI/MR/TA. AI understands many more languages; UI chrome for new pages ("How scoring works", <code>share/flag/priority-projects</code>) is English-only	add keys to <code>js/i18n.js</code> ; add languages to <code>LANGUAGES</code>
Scalable AI platform	🟡 Stateless FastAPI + Postgres; in-memory rate limiter & Telegram state need Redis for multi-instance	swap <code>services/rate_limit.py</code>
Digital Public Good	🟡 MIT licence, no proprietary lock-in, LLM/STT provider switchable, privacy doc	add DPG-Standard docs, open data export
Combine feedback with national demographic data	🗺️ <code>region_indicators</code> table + <code>population</code> feeds L4 and per-100k rates	load real census extracts
...with infrastructure indices and public investment plans	🗺️ same table accepts <code>infra_index</code> , <code>planned_investment</code> ; Priority Projects lists which are missing per hotspot. No dataset is bundled and no formula uses them yet.	extend <code>routers/insights.py</code> scoring
Surface demand hotspots	✅ real-time map + ranked Priority Projects (city × category)	
Recommend high-priority projects to policymakers	🟡 rule-based ranking (severity-weighted demand) + generic project type per category. No cost estimates, no forecasting model.	add cost dataset / optimisation
Measure impact of DPI initiatives	🟡 resolution counts, dispute reopen, status log exist; no before/after outcome metrics yet	
BRICS-wide	❌ India-only data layer (states, schemes, languages). <code>region_indicators.country</code> is BRICS-ready; nothing else is	add Brazil/Russia/China/South Africa locale + scheme data

6. Known simplifications (be upfront in the demo)

- AI filters/classifiers **fail open** — if the LLM key is missing or quota is exhausted, complaints are still accepted (category `other`).
- Score weights are transparent editorial choices, not a statistically validated model.
- "Population" is a default until data is loaded.
- Rate limiting is per-process memory and keys off `X-Forwarded-For`.
- The moderation queue does not yet let an official "dismiss" a citizen report (it is view-only).

Challenge alignment — is the prototype enough?

Challenge (Build with AI: Code for Communities, Track 1, BRICS "Innovation") — build a scalable, multilingual AI platform, designed as a Digital Public Good, that aggregates citizen development requests (voice, text, messaging apps) across linguistic regions, analyses them together with national demographic data, infrastructure indices and public investment plans, surfaces demand hotspots, and recommends high-priority development projects to policymakers across BRICS nations.

Note: the challenge text used here is the statement supplied by the team. The GDG/H2S event page itself could not be consulted while writing this — check it for judging criteria and submission format.

Honest verdict

Strong for the "citizen intake → intelligent prioritisation → official action" half. Partial on the "national data + policy recommendation + BRICS" half. It is a credible prototype if presented as such; the gaps below should be named on stage rather than discovered by judges.

Requirement	Verdict
Voice + text + messaging intake, multilingual	✅ Web, mic, Telegram; Hinglish now handled. ❌ WhatsApp
AI analysis beyond votes	✅ Transparent L1–L5 scoring; explainable per complaint
Hotspots	✅ Live map + ranked Priority Projects
Combine with demographic / infra / investment data	👉 Ingestion pipeline + table built and wired into L4 and hotspot reports; no real datasets loaded
Recommend projects to policymakers	🟡 Rule-based suggestions with evidence links; no costs, no optimisation
Impact measurement	🟡 Status log, disputes, resolved counts
BRICS	❌ India only (schema is country-ready)
Digital Public Good	🟡 MIT, provider-agnostic, privacy doc

What was added in this revision to close gaps

1. **Dataset ingestion** (`region_indicators`, `POST /api/insights/indicators[/csv]`, `python load_indicators.py`) — country/state/city/indicator/value/year/**source** (provenance mandatory).
2. **Population feeds L4** when a dataset covers the city (was a hard-coded 10,000).
3. **Priority Projects** view + `GET /api/insights/priority-projects` — city × category hotspots ranked by severity-weighted demand, per-100k rates when population is known, suggested project type, evidence complaint IDs, and an explicit list of missing datasets. Costs intentionally `null`.
4. Real **Share**, real **citizen Flag** → **moderation queue**, an **L1–L5 explainer page**, Hinglish translation fix, security hardening.

Highest-value next steps (if there were more time)

1. Load a real census extract + one infrastructure index for 2–3 cities; show the before/after effect on L4 and hotspots.
2. WhatsApp Cloud API webhook cloned from `routers/telegram.py`.

3. A second country pack (e.g. Brazil, Portuguese) — `country` field, locale strings, scheme list — to demonstrate BRICS portability.
4. Investment-gap score: `demand_index ÷ planned_investment` per city × category once a plan dataset exists.
5. Impact metrics: median time-to-resolve, dispute rate, per-ward trends.
6. Human-review queue for filter fail-open cases; CSP; Redis rate limiting.
7. Public open-data export (aggregated, anonymised) to strengthen the Digital-Public-Good claim.

Suggested demo script (3 minutes)

1. Submit a Hinglish complaint by mic → show filter, translation, L1–L5 breakdown → open *How scoring works*.
2. Vote on a rival pothole complaint → show it still ranks lower (votes are the weakest signal).
3. Share the link; open it in a private window.
4. Flag it as a citizen → switch to official portal → *Investment Flags* → *Citizen reports*.
5. *Priority Projects*: show ranked hotspots, then the "missing datasets" note — and say plainly what would be loaded next.

Security notes

Audit performed on the final prototype (backend `Backend/`, frontend `js/`). This is a code review + smoke tests, **not** a penetration test.

Fixed in this revision

#	Severity	Issue	Fix
1	Critical	<code>POST /auth/verify-email</code> returned a login token for any <i>already-verified</i> <code>user_id</code> without an OTP, and the public complaint endpoint exposed the author's <code>author_id</code> → anyone could take over a complainant's account.	Endpoint now refuses already-verified accounts; complaint API no longer returns <code>author_id</code> (returns <code>is_author</code> boolean instead).
2	High	No brute-force protection: 6-digit signup/reset OTP (1M values) and passwords could be guessed.	Rate limits: 10 logins/15 min per email, 30/IP; 5 OTP attempts/15 min per account; same for reset; sign-up 10/h per IP.
3	High	Any approved official could change the status of any complaint nationwide.	<code>PATCH /complaints/{id}/status</code> now enforces the official's jurisdiction (ward/city/state; central = nationwide).
4	Medium	Anyone could forge an "Investment Flag" by typing <code>[OFFICIAL_FLAG]</code> in a comment, and post under any name (incl. an official's).	Prefix reserved for officials; signed-in users always post under their own account name.
5	Medium	Stored XSS path: image URL interpolated into an inline <code>onclick</code> (entity-escaping does not protect JS-in-attribute).	Inline JS removed (anchor with <code>rel=noopener</code>), https-only URLs enforced in schema and client; <code>escapeHtml</code> now also escapes quotes.
6	Medium	Unauthenticated LLM-cost endpoints (translate, comments, link-area) without limits.	Per-IP rate limits added.
7	Low	No input length limits (text, comments, notes, images); passwords >72 bytes crash bcrypt.	Pydantic length limits; password max 72.
8	Low	No security headers.	<code>X-Content-Type-Options</code> , <code>X-Frame-Options: DENY</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code> , <code>HSTS</code> (prod).
9	Deploy	<code>render.yaml</code> didn't declare <code>GEMINI_API_KEY</code> / <code>HUGGINGFACE_API_KEY</code> (the actual default providers) → AI silently failed open in production.	Added to the blueprint.

Verified by automated smoke tests (see the session log): anonymous/duplicate/invalid flag → 401/409/422; forged official flag → 403; out-of-jurisdiction status change → 404; `javascript:` image URL → 422; login brute force → 429 after 10; OTP brute force → 429 after 5; already-verified verify-email → 400.

Remaining risks / recommendations

- **Seed accounts** (`ramesh@test.com` / `Test@1234`, `official@test.com` / `Official@1234`) are published in the README. Never seed a public deployment; or delete/rotate them. The seeded official is pre-approved.
- **X-Forwarded-For is trusted** by the rate limiter (`services/rate_limit.py`); behind Render this is normally fine, but a client that can reach the app directly can spoof it. Use a trusted-proxy setting or Redis-backed limits on multi-instance deployments.
- **No CSP**: the frontend relies on inline `onclick` handlers. Moving to `addEventListener` would allow a strict Content-Security-Policy.

- JWTs last 30 days and are not revocable (no logout-everywhere). Consider shorter expiry + refresh.
- AI filter **fails open** by design (availability over strictness). For a production civic system, queue the complaint for human review instead.
- Telegram in-memory state and link codes reset on restart; in `development` the webhook secret is optional — always set `ENVIRONMENT=production` + `TELEGRAM_WEBHOOK_SECRET` when deployed.
- `/docs` (Swagger) is exposed in production; disable if not wanted.
- Anonymous complaints/comments are accepted (by design, per-IP limited). Determined abuse would need CAPTCHA.
- Dependencies were not pinned or scanned; run `pip-audit` before release.
- Secrets: none are committed (`.env` is git-ignored). `google_client_id` is public by design.